

PocketSmart AI: Project Documentation

Executive Summary

PocketSmart AI is a GenAI-powered, cross-platform recommendation and smart budgeting platform. It transforms spending constraints into actionable, category-specific plans for Home Interiors, Party Planning, and Personal Styling/Jewelry.

Managing multi-category budgets across platforms like Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO can be challenging. PocketSmart AI simplifies this by using **Google Gemini 1.5 Flash** via a FastAPI backend to interpret user requirements, budgets, and visual inputs (e.g., outfit photos), delivering structured, cost-optimized recommendations tailored to specific occasions and spaces.

1. System Architecture & Core Technologies

Technology Stack

- **Backend Framework:** FastAPI / Uvicorn server handling asynchronous request routing, session controls, and security.
- **AI Engine:** Google Gemini 1.5 Flash for multimodal processing (text prompts and outfit inspiration images).
- **Authentication:** JWT tokens for session management (/login, /register, /token).
- **Frontend Interface:** Jinja2 templates styled with HTML5 and CSS, providing an interactive dashboard and history tracking.

System Workflow

1. **User Input:** User completes dynamic brief forms on the frontend.
2. **Backend Routing:** FastAPI receives input and routes request to the AI utility service.
3. **AI Processing:** Google Gemini 1.5 Flash evaluates budget constraints, allocations, or multimodal image inputs.
4. **Structured Output:** The AI generates JSON outputs containing itemized costs and e-commerce shopping links (Amazon, Flipkart, Swiggy, Zomato, Tanishq, CaratLane, IKEA).

2. Core Use-Case Scenarios

Scenario 1: Home Interior Planning

- **Inputs:** Total budget, room selection (Living Room, Kitchen, Bedroom), and quantities for lights, ceiling fans, furniture pieces, and dining tables.
- **AI Logic:** Allocates budget proportionally across categories, balancing practical utility and aesthetics.
- **Outputs:** Itemized cost breakdown tables, category allocations, and direct purchase links for IKEA, Amazon, Flipkart, Myntra, and Ajio.

Scenario 2: AI Party Budget Planning

- **Inputs:** Overall budget, guest count, event type (e.g., birthday, corporate, wedding), venue type, and requested services (Catering, Decoration, Entertainment).
- **AI Logic:** Proportionally divides funds across venue hire, catering, decor, and entertainment.
- **Outputs:** Vendor and service recommendations with dynamic search connections to Swiggy, Zomato, BookMyShow, MakeMyTrip, and OYO.

Scenario 3: Jewelry Recommendations for Occasions

- **Inputs:** Target budget, occasion, style choices, and an optional uploaded outfit inspiration image.
- **AI Logic:** Multimodal vision evaluates color schemes, formality, and design elements from the uploaded image.
- **Outputs:** Matching jewelry piece suggestions (bracelets, rings, necklaces, watches) with styling tips and store links for Amazon, Flipkart, Tanishq, BlueStone, and CaratLane.

3. Key Implementation Milestones

Milestone 1: AI Infrastructure Initialization

- Configured Google Cloud AI access and API keys securely in environment variables (.env).
- Initialized the google-genai client SDK with automatic fallback handling.

Milestone 2: Recommendation Utilities Development

- Designed domain-specific JSON prompts ensuring strict output structure and spending compliance.
- Integrated URL encoding engines to generate direct e-commerce search links.
- Added multimodal PIL image handling for outfit analysis in the Jewelry planner.

Milestone 3: Backend Services & Authentication

- Implemented bcrypt password hashing and JWT token authentication.
- Built API endpoints for /home-budget, /party-budget, /jewelry-budget, and user history tracking.
- Configured CORS and cookie session management with automatic cleanup for expired sessions.

Main.py

```

main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
1  import os
2  import uuid
3  import asyncio
4  from datetime import datetime, timedelta
5  from typing import Optional, List, Dict, Any
6
7  from fastapi import (
8      FastAPI,
9      HTTPException,
10     Depends,
11     File,
12     UploadFile,
13     Form,
14     Request,
15     status,
16 )
17 from fastapi.responses import JSONResponse, RedirectResponse, HTMLResponse
18 from fastapi.middleware.cors import CORSMiddleware
19 from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
20 from fastapi.staticfiles import StaticFiles
21 from fastapi.templating import Jinja2Templates
22 from pydantic import BaseModel, EmailStr
23 from passlib.context import CryptContext
24 from jose import JWTError, jwt
25
26 from gemini_utils import (
27     get_home_recommendations,
28     get_party_recommendations,
29     get_jewelry_recommendations,
30 )
31
32 app = FastAPI(title="PocketSmart: AI Budget Planner")
33
34 # Security Configurations
35 SECRET_KEY = os.getenv("SECRET_KEY", "pocketsmart_super_secret_jwt_key_2026")
36 ALGORITHM = "HS256"
37 ACCESS_TOKEN_EXPIRE_MINUTES = 60
38
39 pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
40 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False)
41

```

```

40     oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False)
41
42     # Setup Storage & Directories
43     os.makedirs("static/uploads", exist_ok=True)
44     templates = Jinja2Templates(directory="static/templates")
45     app.mount("/static", StaticFiles(directory="static"), name="static")
46
47     app.add_middleware(
48         CORSMiddleware,
49         allow_origins=["*"],
50         allow_credentials=True,
51         allow_methods=["*"],
52         allow_headers=["*"],
53     )
54
55     # In-Memory Database and Session Storage
56     users_db: Dict[str, dict] = {}
57     active_sessions: Dict[str, Any] = {}
58     blacklisted_tokens = set()
59     user_recommendations: Dict[str, List[dict]] = {}
60
61
62     # Data Models
63     class UserRegister(BaseModel):
64         username: str
65         email: EmailStr
66         password: str
67
68
69     class Token(BaseModel):
70         access_token: str
71         token_type: str
72
73
74     class UserInDB(BaseModel):
75         username: str
76         email: str

```

```

main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
78
79     class HomeBudgetInput(BaseModel):
80         total_budget: float
81         num_lights: int = 0
82         num_fans: int = 0
83         num_furniture: int = 0
84         num_dining_tables: int = 0
85         has_living_room: bool = False
86         has_kitchen: bool = False
87         has_bedroom: bool = False
88         additional_requirements: Optional[str] = None
89
90
91     class PartyBudgetInput(BaseModel):
92         total_budget: float
93         party_type: str
94         num_guests: int
95         venue_type: Optional[str] = "Home"
96         needs_catering: bool = False
97         needs_decoration: bool = False
98         needs_entertainment: bool = False
99         additional_requirements: Optional[str] = None
100
101
102     # Helper Auth Functions
103     def verify_password(plain_password, hashed_password):
104         return pwd_context.verify(plain_password, hashed_password)
105
106
107     def get_password_hash(password):
108         return pwd_context.hash(password)
109
110
111     def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
112         to_encode = data.copy()
113         expire = datetime.utcnow() + (
114             expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
115         )
116         to_encode.update({"exp": expire})
117         return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
118

```

```

main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
118
119
120 async def get_token_from_request(request: Request) -> Optional[str]:
121     # Check Cookie first, then Auth Header
122     token = request.cookies.get("access_token")
123     if not token:
124         auth = request.headers.get("Authorization")
125         if auth and auth.startswith("Bearer "):
126             token = auth.split(" ")[1]
127     return token
128
129
130 async def get_current_user_optional(
131     request: Request,
132 ) -> Optional[UserInDB]:
133     token = await get_token_from_request(request)
134     if not token or token in blacklisted_tokens:
135         return None
136     try:
137         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
138         username: str = payload.get("sub")
139         if username is None or username not in users_db:
140             return None
141         u = users_db[username]
142         return UserInDB(username=u["username"], email=u["email"])
143     except JWTError:
144         return None
145
146
147 async def get_current_user_required(
148     request: Request,
149 ) -> UserInDB:
150     user = await get_current_user_optional(request)
151     if not user:
152         raise HTTPException(
153             status_code=status.HTTP_401_UNAUTHORIZED,
154             detail="Authentication required",
155         )
156     return user
157
158

```

```

main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
149 ) -> UserInDB:
150
151
152
153
154
155
156 return user
157
158
159 def save_to_history(
160     username: str, rec_type: str, input_summary: dict, result: dict
161 ):
162     if username not in user_recommendations:
163         user_recommendations[username] = []
164
165     rec_id = str(uuid.uuid4())
166     record = {
167         "id": rec_id,
168         "timestamp": datetime.now().strftime("%b %d, %Y, %I:%M %p"),
169         "recommendation_type": rec_type,
170         "input_summary": input_summary,
171         "result_summary": {
172             "total_budget": result.get("total_budget", 0),
173             "remaining_budget": result.get("remaining_budget", 0),
174         },
175         "full_result": result,
176     }
177     user_recommendations[username].append(record)
178
179
180 # Background Session Cleanup
181 @app.on_event("startup")
182 async def startup_event():
183     async def cleanup_expired_sessions():
184         while True:
185             now = datetime.utcnow()
186             expired = [
187                 uname
188                 for uname, sess in active_sessions.items()
189                 if (now - sess["last_activity"]).total_seconds() > 1800
190             ]
191             for uname in expired:
192                 del active_sessions[uname]
193             await asyncio.sleep(300)
194
195     asyncio.create_task(cleanup_expired_sessions())

```

```
main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
182 async def startup_event():
183     asyncio.create_task(cleanup_expired_sessions())
184
185 # Authentication Routes
186 @app.post("/register")
187 async def register(user: UserRegister):
188     if user.username in users_db:
189         raise HTTPException(
190             status_code=400, detail="Username already registered"
191         )
192     users_db[user.username] = {
193         "username": user.username,
194         "email": user.email,
195         "password": get_password_hash(user.password),
196     }
197     return {"message": "User registered successfully"}
198
199 @app.post("/token")
200 async def login_for_access_token(
201     form_data: OAuth2PasswordRequestForm = Depends(),
202 ):
203     user = users_db.get(form_data.username)
204     if not user or not verify_password(form_data.password, user["password"]):
205         raise HTTPException(
206             status_code=401, detail="Incorrect username or password"
207         )
208     access_token = create_access_token(data={"sub": user["username"]})
209     active_sessions[user["username"]] = {
210         "login_time": datetime.utcnow(),
211         "last_activity": datetime.utcnow(),
212         "token": access_token,
213         "user_data": {},
214     }
215     response = JSONResponse(
216         content={"access_token": access_token, "token_type": "bearer"}
217     )
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
```

```
main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
216 ):
217     response = JSONResponse(
218         content={"access_token": access_token, "token_type": "bearer"}
219     )
220     response.set_cookie(
221         key="access_token",
222         value=access_token,
223         httponly=True,
224         max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
225         samesite="lax",
226     )
227     return response
228
229
230
231
232
233
234 @app.post("/logout")
235 async def logout(request: Request):
236     token = await get_token_from_request(request)
237     if token:
238         blacklisted_tokens.add(token)
239         try:
240             payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
241             username = payload.get("sub")
242             if username in active_sessions:
243                 del active_sessions[username]
244         except JWTError:
245             pass
246     response = RedirectResponse(url="/login", status_code=status.HTTP_303_SEE_OTHER)
247     response.delete_cookie(key="access_token")
248     return response
249
250
251
252
253
254
255
256
257
258
259
260 # UI Page Web Routes
261 @app.get("/", response_class=HTMLResponse)
262 async def home_page(request: Request):
263     user = await get_current_user_optional(request)
264     return templates.TemplateResponse(
265         request, "index.html", {"request": request, "user": user}
266     )
267
268
269
```

```

main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
209 @app.get("/party-planner", response_class=HTMLResponse):
310 async def party_planner_page(request: Request):
311     user = await get_current_user_optional(request)
312     if not user:
313         return RedirectResponse(url="/login", status_code=status.HTTP_303_SEE_OTHER)
314     return templates.TemplateResponse(
315         request, "party_planner.html", {"request": request, "user": user}
316     )
317
318
319 @app.get("/jewelry-planner", response_class=HTMLResponse)
320 async def jewelry_planner_page(request: Request):
321     user = await get_current_user_optional(request)
322     if not user:
323         return RedirectResponse(url="/login", status_code=status.HTTP_303_SEE_OTHER)
324     return templates.TemplateResponse(
325         request, "jewelry_planner.html", {"request": request, "user": user}
326     )
327
328
329 @app.get("/history", response_class=HTMLResponse)
330 async def history_page(request: Request):
331     user = await get_current_user_optional(request)
332     if not user:
333         return RedirectResponse(url="/login", status_code=status.HTTP_303_SEE_OTHER)
334     history = user_recommendations.get(user.username, [])[::-1]
335     return templates.TemplateResponse(
336         request,
337         "history.html",
338         {"request": request, "user": user, "history": history},
339     )
340
341
342 # Recommendation API Endpoints
343 @app.post("/home-budget")
344 async def plan_home_budget(
345     input_data: HomeBudgetInput,
346     request: Request,
347     current_user: UserInDB = Depends(get_current_user_required),
348 ):
349     result = get_home_recommendations(input_data.dict())
350     save_to_history(

```

```

main.py X
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py
271 async def login_page(request: Request):
272     if user:
273         return RedirectResponse(url="/dashboard")
274     return templates.TemplateResponse(request, "login.html", {"request": request})
275
276
277
278 @app.get("/register", response_class=HTMLResponse)
279 async def register_page(request: Request):
280     user = await get_current_user_optional(request)
281     if user:
282         return RedirectResponse(url="/dashboard")
283     return templates.TemplateResponse(request, "register.html", {"request": request})
284
285
286 @app.get("/dashboard", response_class=HTMLResponse)
287 async def dashboard_page(request: Request):
288     user = await get_current_user_optional(request)
289     if not user:
290         return RedirectResponse(url="/login", status_code=status.HTTP_303_SEE_OTHER)
291     recent = user_recommendations.get(user.username, [])[0:-3:]
292     return templates.TemplateResponse(
293         request,
294         "dashboard.html",
295         {"request": request, "user": user, "recent_activity": recent},
296     )
297
298
299 @app.get("/home-planner", response_class=HTMLResponse)
300 async def home_planner_page(request: Request):
301     user = await get_current_user_optional(request)
302     if not user:
303         return RedirectResponse(url="/login", status_code=status.HTTP_303_SEE_OTHER)
304     return templates.TemplateResponse(
305         request, "home_planner.html", {"request": request, "user": user}
306     )
307
308
309 @app.get("/party-planner", response_class=HTMLResponse)
310 async def party_planner_page(request: Request):
311     user = await get_current_user_optional(request)
312     if not user:

```

main.py X

C: > Nivedha > POCKET_SMART_AI(CODES) > main.py

```
357     async def plan_party_budget(  
358         input_data: PartyBudgetInput,  
359         request: Request,  
360         current_user: UserInDB = Depends(get_current_user_required),  
361     ):  
362         result = get_party_recommendations(input_data.dict())  
363         save_to_history(  
364             current_user.username, "Party Planning", input_data.dict(), result  
365         )  
366         return result  
367  
368  
369 @app.post("/jewelry-budget")  
370 async def plan_jewelry_budget(  
371     total_budget: float = Form(...),  
372     occasion: str = Form(...),  
373     preferences: Optional[str] = Form(None),  
374     image: Optional[UploadFile] = File(None),  
375     request: Request = None,  
376     current_user: UserInDB = Depends(get_current_user_required),  
377 ):  
378     image_path = None  
379     filename = None  
380     if image and image.filename:  
381         filename = f"{uuid.uuid4()}_{image.filename}"  
382         image_path = os.path.join("static/uploads", filename)  
383         with open(image_path, "wb") as f:  
384             content = await image.read()  
385             f.write(content)  
386  
387     input_summary = {  
388         "total_budget": total_budget,  
389         "occasion": occasion,  
390         "preferences": preferences,  
391         "image_file": filename,  
392     }  
393  
394     result = get_jewelry_recommendations(input_summary, image_path)  
395     save_to_history(  
396         current_user.username, "Jewelry", input_summary, result  
397     )
```

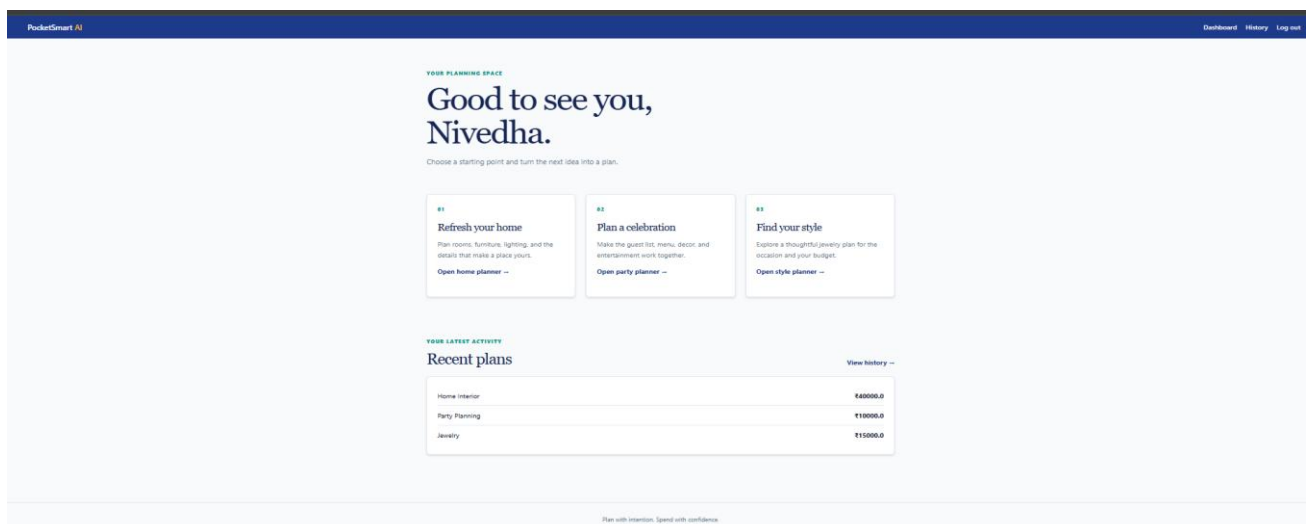
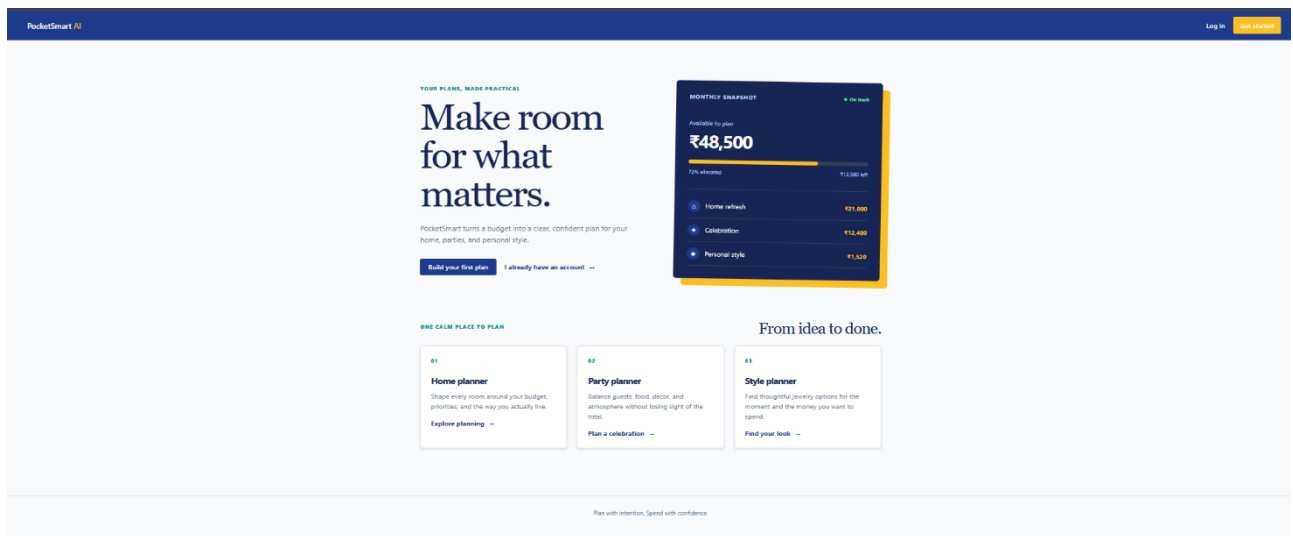
main.py X

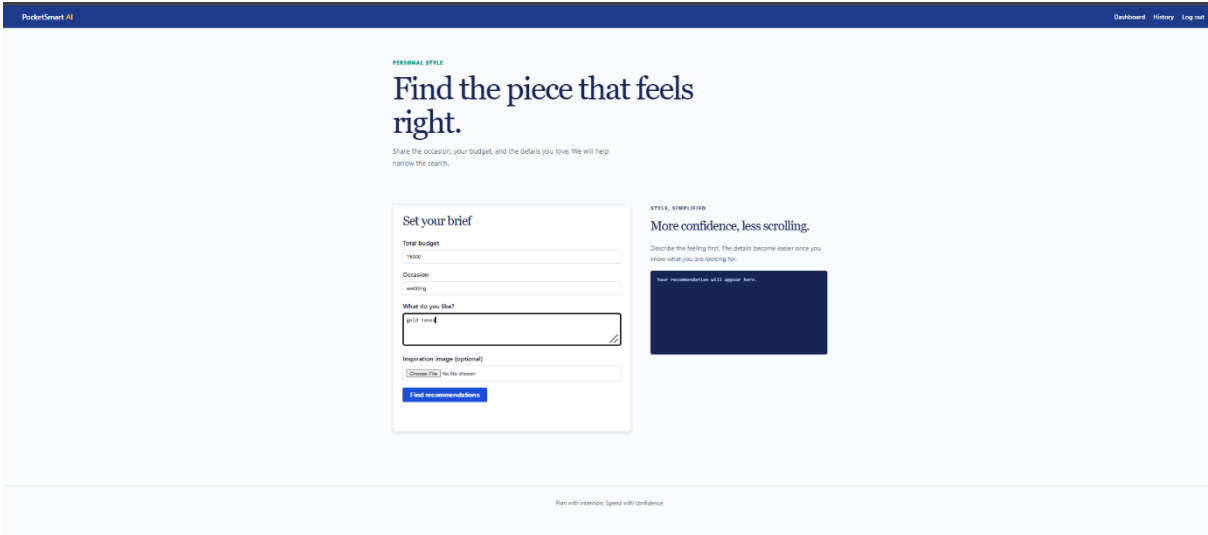
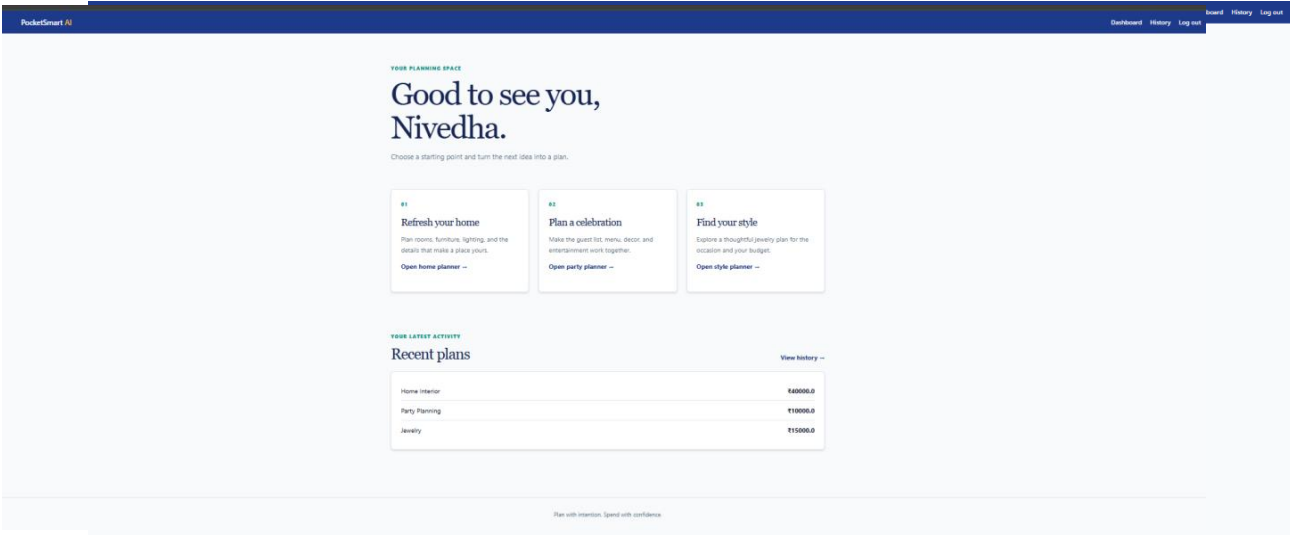
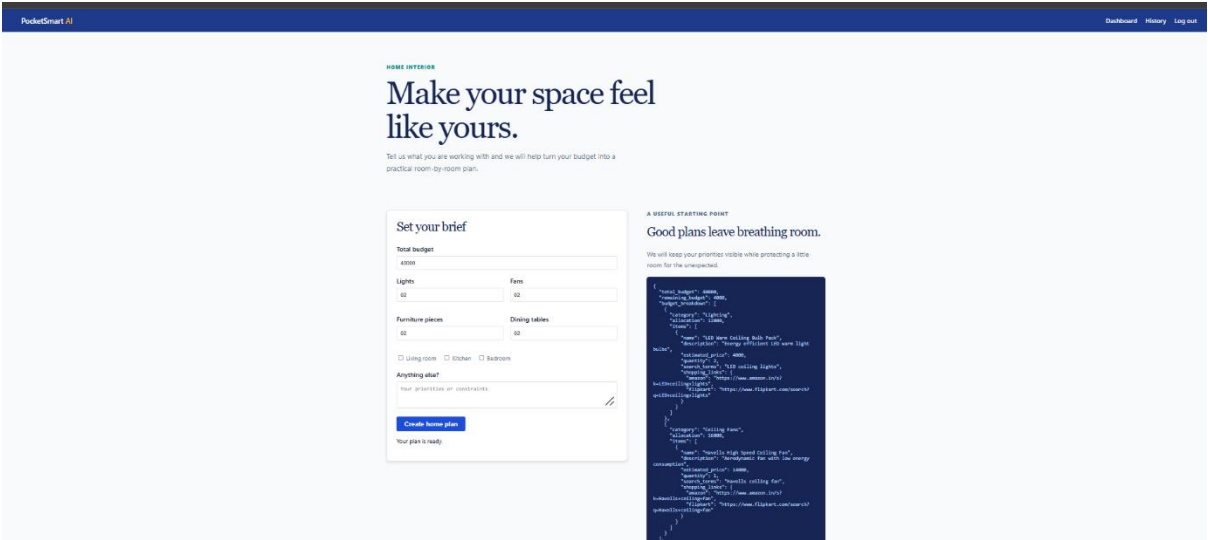
C: > Nivedha > POCKET_SMART_AI(CODES) > main.py

```
377 ):
380     if image and image.filename:
383         with open(image_path, "wb") as f:
384             content = await image.read()
385             f.write(content)
386
387     input_summary = {
388         "total_budget": total_budget,
389         "occasion": occasion,
390         "preferences": preferences,
391         "image_file": filename,
392     }
393
394     result = get_jewelry_recommendations(input_summary, image_path)
395     save_to_history(
396         current_user.username, "Jewelry", input_summary, result
397     )
398     return result
399
400
401 @app.get("/recommendation-details/{rec_id}")
402 async def get_rec_details(
403     rec_id: str,
404     current_user: UserInDB = Depends(get_current_user_required),
405 ):
406     user_recs = user_recommendations.get(current_user.username, [])
407     for rec in user_recs:
408         if rec["id"] == rec_id:
409             return rec
410     raise HTTPException(status_code=404, detail="Recommendation not found")
411
412
413 if __name__ == "__main__":
414     import uvicorn
415
416     print("Starting PocketSmart: AI Budget Planner...")
417     uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)
```

Milestone 4: UI Development & System Testing

- Developed clean, responsive forms and user dashboards using Jinja2 templates and CSS.
- Validated recommendations across spending targets, ensured JSON parsing fallback safety, and optimized response speed.





A USEFUL STARTING POINT

Good plans leave breathing room.

We will keep your priorities visible while protecting a little room for the unexpected.

```
{
  "total_budget": 40000,
  "remaining_budget": 4000,
  "budget_breakdown": [
    {
      "category": "lighting",
      "allocation": 12000,
      "items": [
        {
          "name": "LED Warm Ceiling Bulb Pack",
          "description": "Energy efficient LED warm light bulbs",
          "estimated_price": 4000,
          "quantity": 2,
          "search_terms": "LED ceiling lights",
          "shopping_links": [
            {
              "amazon": "https://www.amazon.in/s?k=LEDceilinglights",
              "flipkart": "https://www.flipkart.com/search?q=LEDceilinglights"
            }
          ]
        }
      ]
    },
    {
      "category": "Ceiling Fans",
      "allocation": 16000,
      "items": [
        {
          "name": "Havells High Speed Ceiling Fan",
          "description": "Aerodynamic fan with low energy consumption",
          "estimated_price": 16000,
          "quantity": 1,
          "search_terms": "Havells ceiling fan",
          "shopping_links": [
            {
              "amazon": "https://www.amazon.in/s?k=Havellsceilingfan",
              "flipkart": "https://www.flipkart.com/search?q=Havellsceilingfan"
            }
          ]
        }
      ]
    }
  ],
  "calculation_table": [
    {
      "category": "lighting",
      "items_count": 2,
      "total_cost": 8000,
      "percentage_of_budget": 20
    },
    {
      "category": "Ceiling Fans",
      "items_count": 1,
      "total_cost": 16000,
      "percentage_of_budget": 35
    }
  ],
  "additional_suggestions": [
    "Opt for modular furniture to optimize small spaces.",
    "Look for festive sales on e-commerce platforms to maximize savings."
  ]
}
```

MAKE ROOM FOR JOY

The best parties feel effortless.

Start with the people and the mood. We will help you decide where the budget matters most.

```
{
  "total_budget": 10000,
  "remaining_budget": 1000,
  "budget_breakdown": [
    {
      "category": "catering",
      "allocation": 5000,
      "items": [
        {
          "name": "Party Snack Box & Buffet",
          "description": "Assorted vegetarian & non-vegetarian platters",
          "estimated_price": 4500,
          "quantity": 10,
          "search_terms": "swiggy party catering",
          "shopping_links": [
            {
              "swiggy": "https://www.swiggy.com",
              "zomato": "https://www.zomato.com"
            }
          ]
        }
      ]
    },
    {
      "category": "decoration",
      "allocation": 3000,
      "items": [
        {
          "name": "Party Balloon & Banner Arch",
          "description": "Complete DIY party decor set",
          "estimated_price": 2500,
          "quantity": 1,
          "search_terms": "party decorations set",
          "shopping_links": [
            {
              "amazon": "https://www.amazon.in/s?k=partydecorationsset"
            }
          ]
        }
      ]
    }
  ],
  "venue_suggestions": [
    {
      "name": "Green Meadows Celebration Hall",
      "type": "Banquet",
      "capacity": 25,
      "estimated_cost": 2000,
      "search_terms": "celebration hall venue",
      "shopping_links": [
        {
          "google": "https://www.google.com/search?q=celebrationhallvenue"
        }
      ]
    }
  ],
  "additional_suggestions": [
    "Consider polka-dot style desserts to reduce catering costs.",
    "Use ambient string lights for inexpensive venue decoration."
  ]
}
```

STYLE, SIMPLIFIED

More confidence, less scrolling.

Describe the feeling first. The details become easier once you know what you are looking for.

```
{
  "outfit_analysis": {
    "colors": {
      "color": "#A"
    },
    "style": {
      "style": "#A"
    },
    "formality": {
      "formality": "#A"
    }
  },
  "total_budget": 15000,
  "remaining_budget": 2210,
  "jewelry_recommendations": [
    {
      "item_type": "Earrings",
      "description": "Gold plated Kundan Jhumka Earrings",
      "style": "Vintage Traditional",
      "estimated_price": 6000,
      "search_terms": "gold plated kundan jhumka",
      "shopping_links": [
        {
          "amazon": "https://www.amazon.in/s?k=goldkundanjumka",
          "tanishq": "https://www.tanishq.co.in"
        }
      ]
    },
    {
      "item_type": "Necklace",
      "description": "Choker Necklace Set with Pearls",
      "style": "Royal Pearl",
      "estimated_price": 6750,
      "search_terms": "pearl choker necklace set",
      "shopping_links": [
        {
          "caratlane": "https://www.caratlane.com",
          "flipkart": "https://www.flipkart.com"
        }
      ]
    }
  ],
  "styling_tips": [
    "Mix warm metallic tones with golden-embroidered outfits.",
    "Keep necklines open when wearing statement choker necklaces."
  ]
}
```

YOUR DECISIONS

Plan history.

A simple record of the recommendations you have created.

Jewelry	Sep 30, 2026, 06:06 PM	₹400000.0
Party Planning	Sep 30, 2026, 06:05 PM	₹200000.0
Home Interior	Sep 30, 2026, 06:05 PM	₹15000.0
Jewelry	Sep 30, 2026, 05:24 PM	₹500000.0
Party Planning	Sep 30, 2026, 05:22 PM	₹100000.0
Home Interior	Sep 30, 2026, 05:20 PM	₹150000.0
Home Interior	Sep 30, 2026, 03:21 PM	₹100000.0

4. Conclusion

PocketSmart AI redefines financial decision-making for lifestyle expenses by converting budget limits into structured, actionable plans. Combining FastAPI with Google's Gemini 1.5 Flash model delivers an intelligent assistant that saves time, prevents financial guesswork, and provides personalized choices tailored directly to user budgets.